

Infraestrutura de Software

Implementação 1 — ProcessFlow

Relatório

Beatriz Loyola Gomes de Vasconcelos
blgv@cesar.school

24 de agosto de 2026

1 Objetivo

Este relatório documenta a implementação do **ProcessFlow**, um orquestrador de processos em C capaz de cadastrar tarefas (**task**) e executá-las por meio de processos filhos criados com **fork()/execvp()**. O programa suporta execução sequencial, paralela e em pipe entre tarefas, redirecionamento de entrada/saída (**input/output/append**), troca de diretório de trabalho (**workdir**), execução em background com controle de jobs (**start/jobs/wait**) e dois modos de operação: interativo (prompt **processflow**) e workflow (leitura de um arquivo **.pf**).

2 Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Modo interativo com prompt processflow >	OK	Rodar ./processflow sem argumentos
Modo workflow (.pf), ecoando cada linha antes de processar	OK	./processflow teste.pf ; comparado via diff com -saida.txt esperado
Comando exit em ambos os modos	OK	Testado nas duas formas
Cadastro de tarefas (task <nome> <programa> [args])	OK	task listar /bin/ls -l
run sequential	OK	run sequential tarefa1 tarefa2 tarefa3
run parallel	OK	Teste com sleep 2/sleep 1 em paralelo, tempo total $\approx 2s$
run <tarefa> sem palavra-chave (sequencial implícito)	OK	Exigido pelo teste3 oficial; args[1] tratado como nome de tarefa quando não é keyword
Pipe entre tarefas (run pipe)	OK	run pipe listar ordenar contar → saída 13, equivalente a ls sort wc -l
Redirecionamento (input/output/append)	OK	Teste com sort ; append validado sem sobrescrever conteúdo prévio
workdir <diretório>	OK	chdir() com perror em caso de erro
Background (start), jobs , wait <jobId>	OK	start processamento imprime [1] <PID>; coleta de zumbis via SIGCHLD
Erros fatais (argc incorreto, workflow inexistente)	OK	Testado nos casos-limite (Fase 9)
Erros não fatais (tarefa/programa/arquivo/job/diretório inexistente)	OK	Programa segue processando após mensagem de erro
Linha vazia, múltiplos espaços, CTRL-D, workflow sem exit	OK	5 casos-limite testados sem crash
Processos com exit code $\neq 0$ e paralelos terminando fora de ordem	OK	Tratado sem interromper o orquestrador

3 Como Reproduzir

Compilar:

```
make clean
make
```

Executar (modo interativo):

```
./processflow
```

Executar (modo workflow):

```
./processflow teste.pf
```

Exemplo de sessão:

```
processflow> task listar /bin/ls -l
processflow> run sequential listar
processflow> task ordenar /usr/bin/sort
processflow> task contar /usr/bin/wc -l
processflow> run pipe listar ordenar contar
processflow> start listar
[1] 1234
processflow> jobs
processflow> wait 1
processflow> exit
```

4 Arquitetura (resumida)

Arquivos e responsabilidades:

1. `main.c` → loop principal (interativo e workflow), parsing de linha e dispatcher de comandos (`processar_linha()`).
2. `task.h/task.c` → struct `Task` e operações de cadastro/busca de tarefas, incluindo campos de redirecionamento.
3. `exec.h/exec.c` → execução de processos: `spawn()` (fork+exec+wait), `spawn_async()` (fork sem wait) e `run_pipe()` (N-1 pipes encadeados).
4. `job.h/job.c` → struct `Job` e gerência de jobs em background (criação, busca por PID, marcação de conclusão).
5. `Makefile` → compilação (`all`), limpeza (`clean`), flags `-Wall -Wextra`.

Decisões técnicas:

1. **Array fixo (`char *args[64]`) em vez de lista encadeada** → `execvp` exige vetor contíguo terminado em `NULL` → simplifica o parsing, ao custo de um limite fixo de argumentos.
2. **Separação em módulos (`task`, `exec`, `job`) em vez de tudo em `main.c`** → isola dados de execução → facilita manutenção e reuso (ex.: `spawn_async()` reaproveitado tanto por `run_parallel` quanto por `start`).
3. **Task vs. Job como entidades distintas** → `Task` é definição estática reutilizável, `Job` é uma execução específica com identidade própria (`job_id`, `PID`, `status`) → permite múltiplas execuções em background da mesma tarefa sem conflito.
4. **Coleta de zumbis via handler de `SIGCHLD`** (`waitpid(-1, &status, WNOHANG)` em loop) em vez de depender só de `wait` manual → jobs terminados saem da lista automaticamente → evita acúmulo de processos zumbis.

5 Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estrat.	Nome curto	Contexto	Motivo da troca
S1	Parser + dispatcher em array fixo	Tokenização de linha com <code>strtok</code> , despacho via <code>strcmp</code> em cadeia <code>if/else if</code> . Array fixo (<code>char *args[64]</code>) escolhido por <code>execvp</code> exigir vetor contíguo terminado em <code>NULL</code> .	—
S2	Módulos separados (<code>task/exec</code>)	Separação de responsabilidades: <code>task.c</code> cuida de dados, <code>exec.c</code> cuida de <code>fork/exec/wait</code> .	—
S3	Pipe com N-1 pipes + <code>dup2</code> encadeado	<code>run_pipe()</code> cria todos os pipes antes de qualquer <code>fork</code> ; cada filho faz <code>dup2</code> conforme posição na cadeia e fecha todos os fds antes do <code>exec</code> .	—
S4	Redirecionamento via campos na struct <code>Task</code>	Campos <code>input_file/output_file/append_file</code> ; <code>spawn()</code> abre o arquivo e faz <code>dup2</code> antes do <code>execvp</code> .	—
S5	Dispatcher extraído para <code>processar_linha()</code>	Refatorado para reaproveitar o mesmo parser entre modo interativo e modo workflow.	—
S6	Módulo <code>job.h/job.c</code>	Struct <code>Job</code> (<code>job_id</code> , <code>pid</code> , <code>nome_task</code> , <code>status</code>) espelhando o padrão de <code>task.c</code> .	—
S7	Background via <code>spawn_async()</code> + <code>SIGCHLD</code>	<code>start</code> reaproveita <code>spawn_async()</code> ; zumbis coletados por handler de <code>SIGCHLD</code> .	—

5.2 Diário de Tentativas

#	Quando	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)
1	19/08 (16:07–18:21)	S1	Loop interativo com <code>fgets</code> + prompt <code>processflow></code>	OK	—
2	19/08 (16:07–18:21)	S1	Tokenização com <code>strtok</code> , guardando ponteiros em <code>args[64]</code>	Falha parcial → corrigida	<code>strcpy(args[i], token)</code> em ponteiro não inicializado; corrigido para atribuição direta do ponteiro.
3	19/08 (16:07–18:21)	S2	Struct <code>Task</code> com <code>char *argumentos[64]</code>	Falha parcial → corrigida	Array de tamanho flexível na struct sem alocação manual; trocado para tamanho fixo.
4	19/08 (16:07–18:21)	S2	<code>criar_task()</code> copiando dados para a struct	Falha parcial → corrigida	<code>strcpy</code> em ponteiros não alocados (trocado por <code>strdup</code>); índice sempre 0 por uso de <code>-></code> em array (corrigido para indexação explícita).
5	19/08 (16:07–18:21)	S2	<code>encontrar_task()</code> por nome	Falha parcial → corrigida	<code>strcmp</code> comparando string com struct; retorno por valor em função que devia retornar <code>Task*</code> .
6	19/08 (16:07–18:21)	S2	<code>spawn()</code> : <code>fork+execvp+waitpid</code>	Falha parcial → corrigida	Faltava <code>exit(1)</code> no filho após <code>execvp</code> falhar, duplicando o loop do pai.
7	19/08 (16:07–18:21)	S1/S2	Teste ponta a ponta: <code>task listar</code> + <code>run sequential</code>	OK	—

#	Quando	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)
8	19/08 (16:07–18:21)	S2	Testes de erro (tarefa/programa inexistente, args insuficientes)	OK	—
9	19/08 (16:07–18:21)	S2	Makefile (all/clean, -Wall -Wextra)	OK	—
10	19/08 (16:07–18:21)	S1	run sequential generalizado para N tarefas	Falha parcial → corrigida	Loop usava args[2] fixo em vez de args[i].
11	19/08 (16:07–18:21)	S2	run parallel via spawn_async()	Falha parcial → corrigida	return; em função pid_t sem valor; waitpid chamado com pids[i]==-1 bagunçando espera de outros filhos.
12	19/08 (16:07–18:21)	S2	Paralelismo real (sleep 2/sleep 1)	OK	—
13	23/08 (13:30–20:14)	S3	Primeira versão de run_pipe()	Falha parcial → corrigida	if (pids[i] == 0) com parênteses trocados; else if impedia os dois dup2 nos processos do meio.
14	23/08 (13:30–20:14)	S3/S5	Teste run pipe listar ordenar contar + ligação no dispatcher	OK	—
15	23/08 (13:30–20:14)	S4	Campos input_- file/output_- file/append_file + comandos	Falha parcial → corrigida	Checagem errada de args[1] em vez de args[0], com loop desnecessário; reescrito sem loop.
16	23/08 (13:30–20:14)	S4	spawn(): open() + dup2 antes do execvp	OK	—
17	23/08 (13:30–20:14)	S4	Teste input/output/append	OK	—
18	23/08 (13:30–20:14)	S5	Extração para processar_linha()	Falha parcial → corrigida	continue de loops internos (sequential/parallel) virou return, abortando a função inteira; revertido.
19	23/08 (13:30–20:14)	—	Chaves desbalanceadas em main.c	Falha → corrigida	Faltava fechar main; depois return 0; preso dentro do while(1).
20	23/08 (13:30–20:14)	S5	Modo workflow (argc==2)	OK (após recompilar)	—
21	23/08 (13:30–20:14)	S5	workdir <dir> com chdir()	OK	—
22	23/08 (13:30–20:14)	S6	criar_job() em job.c	Falha parcial → corrigida	Incremento do contador antes de usar como índice causava overflow do array.
23	23/08 (13:30–20:14)	S6	Retorno do job_id criado	Falha parcial → corrigida	Leitura após incrementar contador, apontando para slot não inicializado.
24	23/08 (13:30–20:14)	S6	marcar_job_- concluido(pid_t pid)	Falha parcial → corrigida	Reaproveitou busca por job_id passando pid; corrigido com busca dedicada por pid.
25	23/08 (13:30–20:14)	S6	listar_jobs() antiga	Código morto removido	status_str não inicializado seguido de strcpy.
26	23/08 (13:30–20:14)	S7	Ligação start/jobs/wait no dispatcher	Falha parcial → corrigida	strcmp(args[0], "jobs") sem ==0.
27	23/08 (13:30–20:14)	S7	Teste start com programa inexistente	Comportamento correto	Job criado no fork() antes de saber se execvp funciona — assíncrono, esperado.
28	23/08 (13:30–20:14)	S7	sigchld_handler() com waitpid(-1, &status, WNOHANG)	OK	—

#	Quando	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)
29	23/08 (13:30–20:14)	—	Revisão de erros: <code>run_pipe()</code> sem aviso de falha de <code>fork()</code>	Falha parcial → corrigida	<code>return</code> ; no meio do loop abandonava filhos já criados; corrigido para só avisar e seguir.
30	23/08 (13:30–20:14)	—	Casos-limite: linha vazia, espaços múltiplos, CTRL-D, workflow sem <code>exit</code> , args inválidos, <code>exit</code> code $\neq 0$	OK	—
31	23/08 (13:30–20:14)	—	Testes oficiais <code>teste1/teste2/teste3</code>	Falha → corrigida	<code>teste3</code> usa <code>run escrever</code> sem keyword; dispatcher exigia <code>argc_line>=3</code> . Corrigido com caso implícito (sequencial por padrão).
32	23/08 (13:30–20:14)	—	<code>diff</code> da saída do <code>teste3</code> vs. esperado	Falha parcial → corrigida	<code>stdout</code> do pai fully-buffered ao redirecionar para arquivo, saída dos filhos saía antes do eco das linhas; corrigido com <code>fflush(stdout)</code> após cada <code>printf</code> de eco.
33	23/08 (13:30–20:14)	—	Comparação de <code>teste1/teste2</code>	Falha aparente, não era bug	Esses testes usam <code>stdin</code> redirecionado (modo interativo), não <code>.pf</code> ; validado com <code>./processflow < entrada.txt</code> , saída bateu 100%.

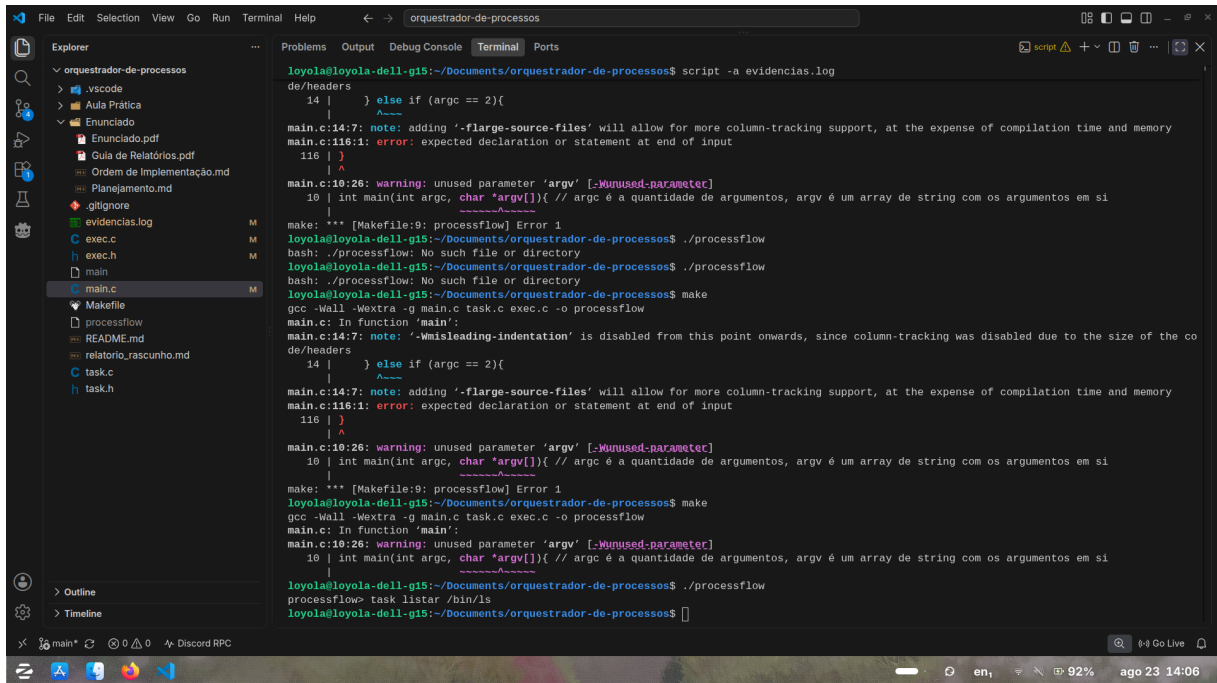
Nota: a coluna "Quando" indica a data (dia/mês) em que cada tentativa foi feita, com base nos timestamps de `Script started/Script done` e nos comandos registrados em `evidencias.log` (gerado com `script -a evidencias.log`). A coluna "Evidência" foi omitida — todas as tentativas têm como evidência esse mesmo arquivo, que registra em ordem cronológica os comandos e saídas de toda a sessão de desenvolvimento.

6 Evidências

O arquivo `evidencias.log` (gerado via `script -a evidencias.log`) registra integralmente os comandos e saídas de todas as sessões de trabalho (19/08 e 23/08), incluindo `date`, `whoami` e `pwd` no início de cada sessão, e é enviado junto com o código-fonte no `.tar`. Os principais pontos de evidência:

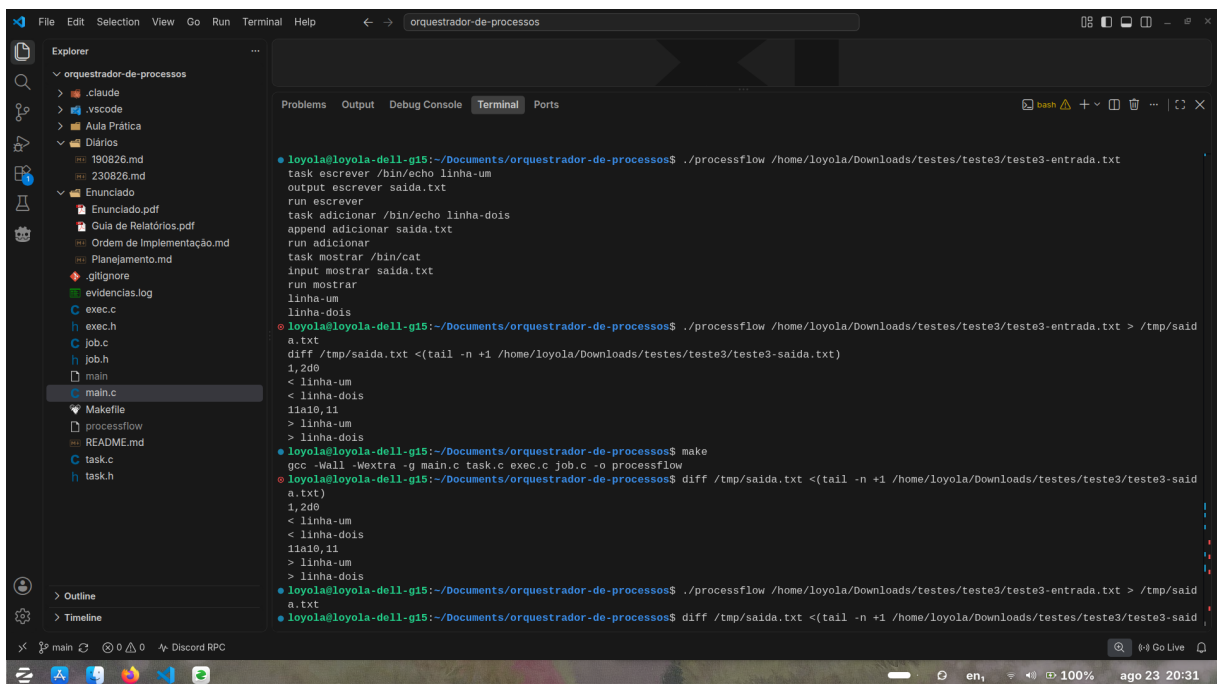
- **Erro principal da semana:** bug de `dup2` nos processos do meio do pipe (tentativa 13), que impedia a leitura e escrita simultâneas por causa de um `else if` incorreto.
- **Validação final:** execução dos três testes oficiais do professor (`teste1`, `teste2`, `teste3`) com `diff` limpo contra as saídas esperadas (tentativas 31–33).
- **Achado de buffering:** diferença de comportamento de `stdout` entre terminal (line-buffered) e saída redirecionada para arquivo (fully buffered), só percebida ao testar com `> arquivo` (tentativa 32).

6.1 Prints



```
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ script -a evidencias.log
de/headers
14 |     } else if (argc == 2){
    |     ^
main.c:14:7: note: adding '-flarge-source-files' will allow for more column-tracking support, at the expense of compilation time and memory
main.c:116:1: error: expected declaration or statement at end of input
116 | }
    | ^
main.c:10:26: warning: unused parameter 'argv' [-Wunused-parameter]
10 | int main(int argc, char 'argv[]){ // argc é a quantidade de argumentos, argv é um array de string com os argumentos em si
    |                          ^
make: *** [Makefile:9: processflow] Error 1
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ ./processflow
bash: ./processflow: No such file or directory
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ ./processflow
bash: ./processflow: No such file or directory
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ make
gcc -Wall -Wextra -g main.c task.c exec.c -o processflow
main.c: In function 'main':
main.c:14:7: note: '-Wmisleading-indentation' is disabled from this point onwards, since column-tracking was disabled due to the size of the co
de/headers
14 |     } else if (argc == 2){
    |     ^
main.c:14:7: note: adding '-flarge-source-files' will allow for more column-tracking support, at the expense of compilation time and memory
main.c:116:1: error: expected declaration or statement at end of input
116 | }
    | ^
main.c:10:26: warning: unused parameter 'argv' [-Wunused-parameter]
10 | int main(int argc, char 'argv[]){ // argc é a quantidade de argumentos, argv é um array de string com os argumentos em si
    |                          ^
make: *** [Makefile:9: processflow] Error 1
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ make
gcc -Wall -Wextra -g main.c task.c exec.c -o processflow
main.c: In function 'main':
main.c:10:26: warning: unused parameter 'argv' [-Wunused-parameter]
10 | int main(int argc, char 'argv[]){ // argc é a quantidade de argumentos, argv é um array de string com os argumentos em si
    |                          ^
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ ./processflow
processflow> task listar /bin/ls
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$
```

Figura 1: Erro principal da semana: chaves desbalanceadas em main.c após edições manuais (main.c:116:1: error: expected declaration or statement at end of input, tentativa #19), seguido da correção e recompilação bem-sucedida.



```
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ ./processflow /home/loyola/Downloads/testes/teste3/teste3-entrada.txt
task escrever /bin/echo linha-um
output escrever saida.txt
run escrever
task adicionar /bin/echo linha-dois
append adicionar saida.txt
run adicionar
task mostrar /bin/cat
input mostrar saida.txt
run mostrar
linha-um
linha-dois
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ ./processflow /home/loyola/Downloads/testes/teste3/teste3-entrada.txt > /tmp/said
a.txt
diff /tmp/saida.txt <(tail -n +1 /home/loyola/Downloads/testes/teste3/teste3-saida.txt)
< linha-um
< linha-dois
11a10,11
> linha-um
> linha-dois
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ make
gcc -Wall -Wextra -g main.c task.c exec.c job.c -o processflow
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ diff /tmp/saida.txt <(tail -n +1 /home/loyola/Downloads/testes/teste3/teste3-said
a.txt)
1,2d0
< linha-um
< linha-dois
11a10,11
> linha-um
> linha-dois
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ ./processflow /home/loyola/Downloads/testes/teste3/teste3-entrada.txt > /tmp/said
a.txt
loyola@loyola-dell-g15:~/Documents/orquestrador-de-processos$ diff /tmp/saida.txt <(tail -n +1 /home/loyola/Downloads/testes/teste3/teste3-said
```

Figura 2: Validação final: diff entre a saída do teste3 oficial e o resultado esperado, limpo após a correção do fflush(stdout) no eco de linhas do modo workflow (tentativas #31-32).

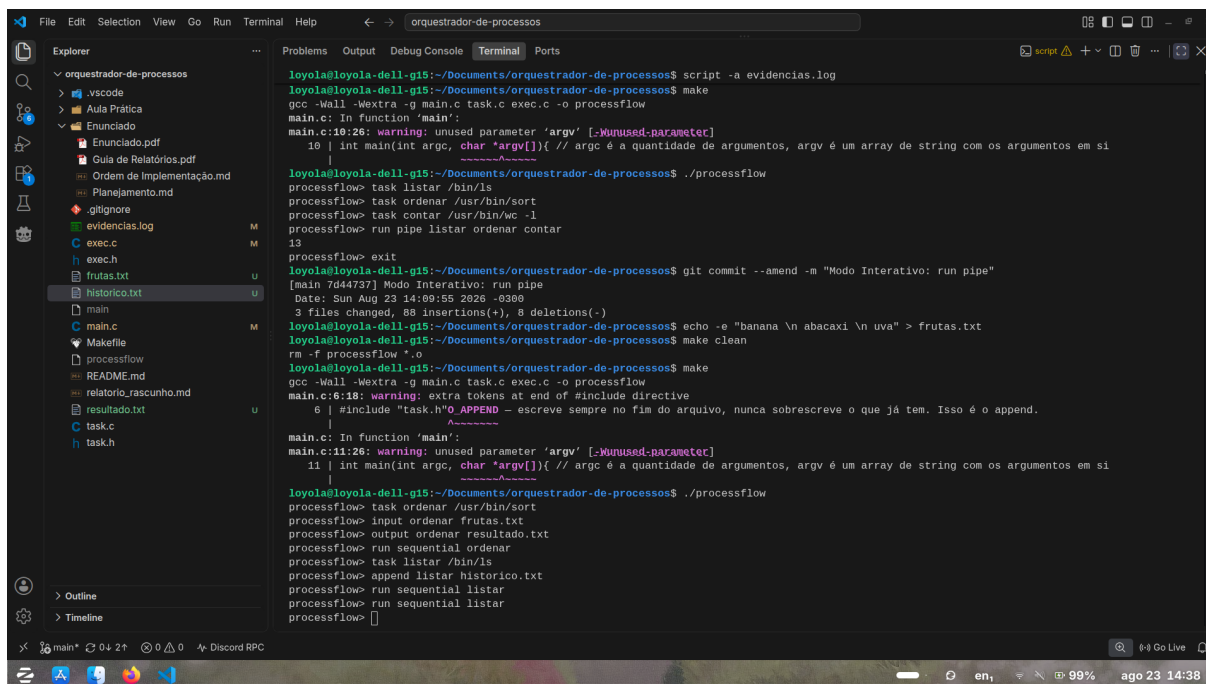


Figura 3: Teste ponta a ponta de `run pipe listar ordenar contar` (saída 13) e de redirecionamento (`input/output/append`), confirmando pipe e redirecionamento funcionando em conjunto.

7 Uso de IA

Onde usei IA: utilizada como mentora/revisora de código durante todo o desenvolvimento — não escreveu a lógica principal do programa, mas apontou bugs (ponteiros não inicializados, tipos incompatíveis entre struct e string, erros de escopo em refatoração), explicou conceitos de C (`strtok`, `fork/exec/wait`, `dup2`, `SIGCHLD`) e ajudou a montar o Makefile.

Prompts principais: pedidos de revisão de trechos de código com bug suspeito; explicações sobre manipulação de descritores de arquivo e pipes; dúvidas conceituais sobre diferença entre buffering de terminal e de arquivo.

O que validei manualmente: cada trecho corrigido foi testado rodando `./processflow` com casos reais (tarefas de teste, pipes, redirecionamento, background) antes de prosseguir para a próxima etapa; todos os testes oficiais do professor foram validados com `diff` contra as saídas esperadas.

8 Reflexão Final

O desenvolvimento revelou dois padrões de bug recorrentes que se repetiram em módulos diferentes: (1) ponteiros não inicializados usados com `strcpy` em vez de alocação com `strdup`, e tipos incompatíveis entre `struct` e `string` em comparações/retornos; (2) inversão da ordem "usar índice, depois incrementar" vs. "incrementar, depois usar índice", que apareceu tanto em `criar_task` quanto em `criar_job`. Isso indica que, com mais tempo, valeria a pena extrair um padrão utilitário comum para criação de entidades em arrays estáticos (`tasks/jobs`), reduzindo a chance de repetir o mesmo erro. A separação em módulos (`task`, `exec`, `job`) e a extração do dispatcher em `processar_linha()` foram decisões acertadas, pois permitiram reaproveitar a mesma lógica entre modo interativo e workflow, e reaproveitar `spawn_async()` entre `run_parallel` e `start`. O bug mais custoso foi o de buffering de `stdout` ao redirecionar a saída para arquivo, só descoberto ao comparar a saída do `teste3` via `diff` — lição de que testes manuais no terminal não são suficientes e é preciso sempre testar também com saída redirecionada.

9 Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e <code>evidencias.log</code> foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10 Se eu tivesse mais 2 horas

Investiria em: (1) tratamento mais robusto de limites fixos (`args[64]`, `tasks[64]`, `jobs[64]`), avisando o usuário ao invés de falhar silenciosamente ao estourar; (2) organização do `evidencias.log` em seções por sessão para facilitar a leitura cruzada com a tabela de tentativas.